LegalOS: how it works, technically

For the reviewer who asks how we built this. Each section gives the industry term, what it means in plain words, and how we used it. Edge cases and a vocabulary sheet at the end.

1. Architecture in one picture

• **Modular monolith with schema-per-workspace.** One FastAPI backend, but each workspace (contracts, notices, vendors, compliance, documents) has its own Postgres schema, its own engine folder and its own tests. Plain words: five mini-products that share login, the calendar and the AI layer, but cannot corrupt each other's data.

• **Stack.** Python 3.11, FastAPI, async SQLAlchemy 2 with asyncpg, Alembic migrations, PostgreSQL 15, Qdrant vector database, Google Gemini via google-genai, sentence-transformers for embeddings and reranking. Frontend: React 18, TypeScript, Vite, Tailwind, TanStack Query, Radix. Everything runs in Docker Compose.

• **Multi-tenant.** Every row carries an organisation id and every query is filtered by it; JWT sessions; roles (owner, CA, reviewer). Plain words: two companies on the same server can never see each other's files.

• **Background jobs.** An upload returns immediately; analysis runs as a background task with stages (reading, finding clauses, checking Indian law, scoring). The UI polls the stage. A restart resumes interrupted jobs.

• **604 automated tests** run against a separate test database with a fake AI provider, so the suite never spends AI quota and never depends on the network.

2. The AI layer (shared by all workspaces)

• **Structured output.** We never ask the model for free text. Every call sends a JSON schema (a Pydantic model) and the model must fill it. Plain words: the AI fills a form, it cannot ramble.

• **Typed errors, no silent defaults.** Quota exhausted, auth failure, invalid JSON and model missing are different exception classes. The UI shows the real failure. We never show a risk score of zero or an empty draft as if it were a result.

• **Key rotation and model fallback.** Free Gemini keys allow 20 requests per model per day. The client rotates across four keys and three models, remembers which pairs are exhausted (persisted to disk), honours Google's retry hints and queues bursts instead of failing.

• **Prompt-versioned cache.** Every call is cached on disk keyed by model, prompt, schema and prompt version. The same document analysed twice costs nothing; changing a prompt invalidates only its own cache.

• **Quote verification.** The core trust mechanism. Whenever the model claims something it must return the verbatim quote. We search for that quote in the document text (Unicode-normalised, Hindi-aware, fuzzy for long quotes). If it is not found, the finding is flagged unverified and excluded from the score. Plain words: the AI shows its work and we check it.

• **JSON repair.** Truncated or malformed output gets one repair pass; otherwise it fails loudly. Output token caps are sized from the schema, with extra room for Hindi, because JSON escapes each Devanagari letter as six characters.

3. Contracts: the review pipeline

• **Parse** PDF (pdfplumber, pdfium), DOCX (python-docx with tables and auto-numbering restored), images and scanned PDFs (Gemini vision OCR in batches of 8 pages), plain text.

• **Segment** into clauses using numbering patterns (1., 1.1, Section, Clause, Hindi dhara and khand headings, Devanagari digits). Each segment keeps its page.

• **Pass 1, facts.** One structured call extracts parties, type (29 types), value, dates, termination terms and governing law. If the user's company is not a party, the model names the likely client (the party that pays, rents or borrows) and the review is done from that side. This fixed a real bug: a lease reviewed from the landlord's side found 3 issues; from the tenant's side it finds 20.

• **Pass 2, risks.** One structured call walks a type-specific checklist (lease: lock-in, escalation, deposit; employment: non-compete, notice asymmetry; SaaS: liability cap, auto-renewal, data deletion). Every finding carries severity, why it matters, what to ask for, replacement wording and the verbatim quote with its segment id.

• **Rules engine (deterministic).** 24 Indian-law rules in code, each with an act and section: MSME 45-day payment (MSMED Act s.15), non-compete after employment (Contract Act s.27), penalty versus liquidated damages (s.74), interest above 18 or 24 percent, blank cheques as security, director personal guarantee, lock-in over 24 months without exit, deposit refund over 60 days, rent escalation over 10 percent, foreign governing law or arbitration seat, stamp duty, no GST clause, retention of original certificates, auto-debit without a cap. A rule fires only when it locates the clause text; that sentence is its evidence.

• **Recall sweep.** If the AI pass returns fewer than 5 verified findings on a contract of 8 or more clauses, one extra call walks the checklist topic by topic. **Long contracts** are split into two parts with one extra call and merged; the coverage note says what was read.

• **Scoring.** Deterministic: critical 35, high 20, medium 10, low 3 points; bands low under 20, medium under 50, high at 50 and above. Unverified findings never count. The UI shows every contribution under How the score is built.

• **Chat over the contract.** One call with the ranked findings and the relevant clauses; answers cite verified quotes; enforceability questions get the matching legal note.

4. Notices: the response desk

• **A catalogue of 45 notice types** in code: GST (ASMT-10, DRC-01 under s.73, 74 and 74A, DRC-07, REG-17, GSTR-3A), income tax (142(1), 143(2), 148, 156, 271, TDS defaults), PF and ESI, labour, factory, pollution board, FSSAI, legal metrology, municipal, bank and SARFAESI s.13(2), MSME Samadhaan, arbitration s.21, civil and criminal summons, police summons, insurance repudiation, RERA, consumer, IP, defamation, tenancy, employment, contract breach. Each entry has the meaning, the consequence chain if ignored, the act and section, documents to gather, who handles it (owner, CA, lawyer, lawyer required) and whether the right response is a letter or an action plan.

• **Deadline engine (deterministic).** Candidates: a date stated in the notice (verified by quote), a period in the notice counted from receipt or the notice date, a hearing date, and a statutory window only where a statute fixes one (cheque bounce 15 days under NI Act s.138, consumer 30 days, SARFAESI 60 days, income-tax demand 30 days under s.220, ASMT-10 30 days under rule 99). The earliest wins and the basis is shown. Indian date formats, Hindi month names and Devanagari digits are parsed. Missing dates are asked for, never guessed.

• **One extraction call** returns type, parties, allegations, amounts, key quotes (deadline, amount, demand), actions and consequences; the quotes are verified against the text. Hindi, Marathi and Gujarati notices give English facts plus a local-language summary.

• **Reply drafting.** Per-type structure: reference, a without-prejudice line where right, numbered paragraphs answering each allegation, reservation of rights, enclosures. Only the notice's facts and the company profile are used; everything else is a CONFIRM blank. For types where a letter is wrong (court summons, 143(2) e-proceedings, DRC-07 appeal) the output is an action plan and the UI says so. A lawyer-review gate blocks final until a lawyer confirms.

• **Lawyer brief** is built from stored fields with no AI call. The deadline is pushed to the shared compliance calendar.

5. Documents: retrieval-augmented generation (RAG)

• **RAG in plain words:** we do not let the AI answer from memory. We find the right passages in the company's own files first, hand only those to the model, and make it cite them.

• **Ingest.** Parse PDF, scanned PDF, DOCX, XLSX, CSV, images, text. **Structure-aware chunking:** heading hierarchy becomes a section path (4 > 4.2), lists stay with their lead-in line, table rows carry their header, page headers and footers are stripped, splits happen at sentence ends. Each chunk is stored in Postgres with full-text indexes and embedded into Qdrant with a multilingual MiniLM model (384 dimensions).

• **Hybrid retrieval.** Keyword search (Postgres full-text) plus vector search (Qdrant) plus an exact-token pass for codes, amounts, dates and clause numbers, fused with Reciprocal Rank Fusion. A query planner (one AI call) rewrites the question, splits multi-part questions and produces English and Hindi variants. A document-routing step compares the question with each document's profile so the right file answers. A **cross-encoder reranker** then re-scores the top 30 passages against the question. Per-document diversity caps keep a second relevant document visible.

• **Answer.** One structured call: a direct answer, supporting bullets, not-covered and sources-disagree fields, every sentence with a citation marker. Citations are verified verbatim; confidence is computed from the verified share and reranker agreement, never asked of the model. The answer language is locked to the script of the question (Hindi, Hinglish, English).

• **Intents.** Lookup (search and cite), overview (map-reduce across the whole document for summaries), conversation (greetings and what documents do I have, answered with no AI call).

• **Measured.** A labelled evaluation set of 33 questions over 9 synthetic documents (tables, Hindi, exact codes, multi-document, follow-ups): the right passage is the top result for 32 of 33 and within the top 8 for all 33. Before this hardening it was 24 of 33.

6. Vendors and compliance

• **Identifiers.** Pure functions validate GSTIN (state code, embedded PAN, entity letter, mod-36 check digit), PAN, TAN, Udyam, CIN, LLPIN, IFSC, IEC, FSSAI, ESIC and PF formats. The GSTIN is validated as the user types, in the browser, before any network call.

• **Live registry.** A pluggable provider (Sandbox.co.in today) with typed failures; results cached for 24 hours per GSTIN; a failed lookup is never shown as a valid supplier. The verdict is deterministic: cancelled or suspended means do not pay; a name or PAN mismatch or non-filing means check before paying; live and active means safe; offline only means looks valid but not verified live.

• **Compliance rules engine.** 30 rules in YAML, each with a schedule, the act and section, the penalty text and per-day late-fee data; conditions on entity type, state, registrations, turnover and the composition scheme. The calendar regenerates when the profile changes (an idempotent sync keeps user edits). A lint test checks every rule has a source and a valid schedule and that no two collide. Contract dates, notice deadlines and document expiries land on the same calendar.

7. Edge cases the reviewer may probe, with honest answers

Q. How do you stop the AI from hallucinating?

A. Three layers: structured output (it fills a schema, not prose); mandatory verbatim quotes that we verify against the text; deterministic rules for the law. Anything unverified is flagged and excluded from the score. The UI shows found word for word per citation.

Q. What if the document is scanned or a photo?

A. Page-wise OCR through Gemini vision in batches of 8 pages; several photos become pages in order. The text then goes through the same pipeline. OCR costs one extra call per batch.

Q. What if the contract is 80 pages?

A. It is split at clause boundaries into two parts; part two is reviewed in one extra call and merged. Absence rules (no liability cap, no dispute clause) are skipped on partially read contracts because absence cannot be claimed from a part. The coverage note says exactly what was read.

Q. What if the notice has no date, or a Hindi date?

A. Dates like 12.10.2026, 12-10-26, 12th October 2026, Hindi month names and Devanagari digits are parsed. If no date is found, the UI asks for the date on the notice and the received date and recomputes. We never guess a deadline.

Q. Which deadline do you pick when the notice says 15 days but the law says 30?

A. The earliest. The strip lists every candidate with its basis (stated in the notice and verified, counted from the law with the section, a fixed hearing date) and marks the one chosen.

Q. Hindi documents and questions?

A. Chunking, keyword search and quote verification are Unicode-aware; embeddings are multilingual; the planner searches both the original Devanagari words and an English version; answers follow the script of the question.

Q. Why not just use ChatGPT?

A. It cannot query the GST registry, cannot build a filing calendar from a company profile, cannot verify its own quotes against the file, does not push deadlines to a calendar a CA shares, and keeps no audit trail. Our value is the verification and the workflow, not the model.

Q. What happens when the AI service is down or out of quota?

A. The job fails with a typed error; the screen says so and offers retry; nothing fake appears. Keys and models rotate automatically first. Everything already analysed keeps working because results are stored, not recomputed.

Q. Is this legal advice?

A. No. Decision support. Every legal note shows its source and some are marked being verified. Replies cannot be finalised without lawyer review. We say this on screen.

Q. Where is the data? Privacy?

A. Local Postgres and Qdrant, per organisation, with role-based access. Only the document being analysed is sent to the AI provider. Uploads are validated by magic bytes, size-limited and de-duplicated by content hash.

Q. How do you measure accuracy?

A. Documents: the labelled retrieval set (hit at 1 is 96 percent, hit at 8 is 100 percent). Contracts and notices: the verified-citation rate shown live per result, deterministic rule tests (positive and negative for every rule), and 604 automated tests overall. A RAGAS-style faithfulness number is the next step.

Q. What is not done?

A. A fine-tuned Indian legal model (InLegalBERT) is future work; Professional Tax only for Karnataka; MCA registry lookups need paid access; some legal notes await a lawyer's confirmation and are marked so; Hindi UI.

Q. Cost per company?

A. On the free tier a contract costs 2 to 3 AI calls, a notice 2, a question 2. On paid Gemini pricing that is a few rupees per document. Embeddings and reranking run locally at zero marginal cost.

Vocabulary: RAG (retrieve passages, then answer from them) | embeddings (text turned into vectors so meaning can be compared) | vector database (Qdrant stores those vectors) | hybrid search (keywords plus vectors) | RRF (merges ranked lists) | cross-encoder reranker (reads question and passage together, more precise) | structured output (the AI fills a JSON schema) | quote verification (we check the AI's quote exists in the file) | deterministic rules (code, not AI) | idempotent sync (re-running does not duplicate) | multi-tenant (data separated per company) | OCR (image to text).